DECISION BRIEF · v2 · NEON

Vhagar POS

Scan-to-Bill Stock & Billing System for Bharat Tex 2026

Phone/Tablet QR Scanning | Live Cash Billing | Real-time Inventory | Next.js + Neon on Vercel

Prepared for **LD Group · Vhagar Clothing** | Date **26 June 2026** | Go-live **02 July 2026** | Runway **6 days**

1. The Goal in One Line

At the Vhagar booth in **Bharat Tex 2026**, a staff member scans a garment's QR tag with a phone/tablet → it drops into a cart → the price is confirmed (editable for bargaining) → a **cash bill** is generated and **stock decrements live**. No spreadsheets, no manual stock counts, no double-selling the last piece.

103	318	677	6
Styles	Variants (style×size)	Physical pieces = QR labels	Days to go-live

Data foundation is **built, merged and loaded** (per-size stock + price/name/colour). Only **4 styles still need a confirmed price** — otherwise the listed default applies.

2. The Stack — Decided NEON

A **custom web app** on the team's own stack, so it fits the existing toolchain and stays free to run:

Frontend

Next.js 14 PWA

Installs to the home screen, built-in camera QR scanner, big booth-friendly buttons.

Hosting

Vercel

One-push deploy, automatic HTTPS (required for the camera), native Neon integration.

Database

Neon Postgres

Serverless Postgres, scales to zero, instant provisioning, connected to Claude via MCP for the build.

> **Why Neon (and not Supabase).** The plan originally pointed at Supabase, but its free plan caps at **2 active projects** and both slots were already taken (the SCOT dashboards). Rather than pay or pause an existing app, we moved the database to **Neon** — serverless Postgres on a genuinely free tier, provisioned in seconds, with a Vercel-native integration and a Claude MCP connection that lets the build run schema + seed directly. Net result: **$0, fully isolated, zero friction.**

Locked decision	Detail
Bill type	**Simple cash bill, no GST.** Subtotal − discount = total.
Scanning	**QR codes** (cameras read QR off fabric tags far better than 1D barcodes). The QR encodes the `variant_sku` (e.g. `VH-EP10-L`).
Pricing	**Manual price per sale** — the listed price pre-fills as an editable default (handles bargaining).
Connectivity	**Cloud over a dedicated 4G hotspot**, not venue wifi.
No-oversell guard	Database **physically cannot go negative**; checkout is one atomic transaction that rolls back rather than sell stock you don't have.

3. How It Works — Architecture

The flow (per sale)

- **Scan** — phone camera reads the garment's QR (encodes `STYLE-SIZE`, e.g. `VH-EP10-L`).
- **Cart** — variant drops in; price pre-fills as an *editable* default for bargaining.
- **Bill** — confirm → simple cash bill (no GST), shareable/printable.
- **Stock** — that variant decrements by 1, instantly, for everyone.

The stack at runtime

- **Next.js PWA on Vercel** serves the app over HTTPS.
- **Neon Postgres** holds catalog, stock, sales — accessed only server-side.
- **Live stock** stays fresh by lightweight polling (Neon has no realtime service; for a few booth devices, polling is simpler and rock-solid).
- **4G hotspot** for connectivity; the app tolerates flaky signal.

Tagging — QR stickers on every piece

Each of the **677 physical pieces** gets a printed QR sticker encoding its `variant_sku`, with the style name, size, colour and price in human-readable text underneath (so a sale still works if a scan fails). Pieces of the same style+size carry an identical QR — scanning simply sells one of that size. Stickers go on the **hang tag / polybag / care-label** (not bare fabric) and are printed in-house on the **Kores Endura label printer**.

Screens to build

Screen	Purpose
Scan & Sell	Live camera scan → cart → price edit → checkout. The booth's main screen.
Bill	Generate cash bill, share/print, finalize sale & decrement stock.
Stock	Live inventory by style/size; low-stock highlight.
Sales Log	Running list of the day's sales & total; CSV export (our backup habit).
Admin + QR Generator	Set the 4 missing prices; generate & print all 677 QR labels.

4. Timeline — 6 Days to Go-Live

Thu 26 Jun
Day 0 — today

Foundation & database. DONE Neon chosen, schema + atomic checkout built, catalog seeded (103/318/677), Next.js project scaffolded.

Fri 27 Jun
Day 1

Core app. Wire Neon URL, deploy skeleton to Vercel, build Scan & Sell + cart with live camera scanning.

Sat 28 Jun
Day 2

Billing engine. Cash bill generation, atomic stock decrement, sales log, polled stock sync across devices.

Sun 29 Jun
Day 3

Labels + admin. QR label generator → print all 677 on the Kores printer. Set 4 missing prices. Returns/void handling.

Mon 30 Jun
Day 4

Hardening. Offline/poor-signal resilience, 4G hotspot test, multi-device test, edge cases & polish.

Tue 1 Jul
Day 5

Dress rehearsal. Full dry run at booth setup: stick labels, train staff, simulate a busy hour, fix what breaks.

Wed 2 Jul
Day 6 — GO-LIVE

Live at Bharat Tex. Buffer for last-mile fixes; monitor first real sales.

Status: the database layer is finished ahead of schedule. The working core (scan → bill → stock) is realistically **2–3 focused days**; the rest is hardening + rehearsal — exactly what de-risks a live event. Comfortable slack remains.

5. Risks & How We Cover Them

Risk	Mitigation
DB idle / cold start mid-event	Neon compute auto-suspends when idle and **auto-resumes on the next query** (a brief cold start, not a manual restore). A busy booth keeps it warm; a tiny keep-alive ping covers quiet spells.
Data loss / corruption	Neon keeps point-in-time history for restore within the plan's retention window. We also **export sales to CSV nightly** as a belt-and-suspenders backup.
Weak 4G signal at booth	Dedicated hotspot; PWA caches the app shell; the cart survives a failed request so staff can retry.
Two staff sell the last piece	Atomic server-side decrement with a non-negative stock constraint; the second sale cleanly rolls back.
Camera won't scan a tag	QR (not barcode) + human-readable SKU on the sticker + manual SKU/name search fallback.

6. Where We Are & Next Steps

- **Done:** Neon chosen · schema + atomic checkout · catalog seeded (103/318/677) · Next.js scaffold + kickoff prompt.
- **You:** paste the Neon `DATABASE_URL` into `.env.local`; send confirmed prices for the 4 styles (`VH-KN100`, `VH-PLV5197A`, `VH-X5C28A`, `VH-Y5A183B`).
- **Build:** run schema + seed → Scan & Sell → Bill → Stock → Sales Log → Admin/QR labels → deploy to Vercel → booth dress rehearsal.